

形式语言与编译

计算机 2303 尤颢辰

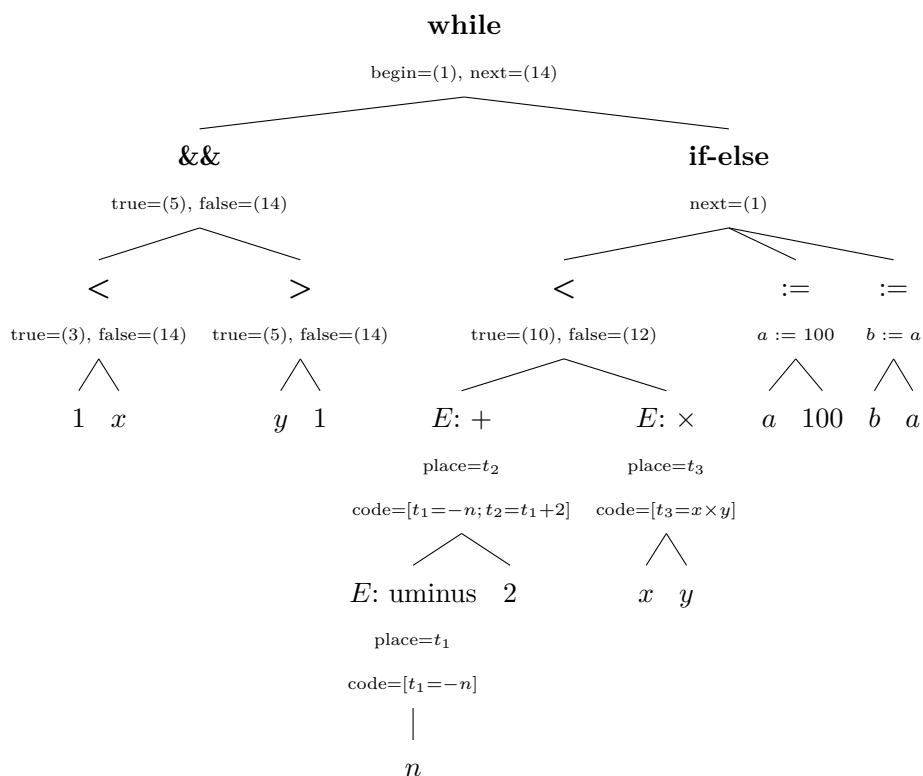
2026 年 6 月 10 日

1 习题 10.2(1)

1.1 三地址代码

```
1 (1)  if 1 < x goto (3)      ; B1 为真 → 接着测 B2 (短路)
2 (2)  goto (14)             ; B1 为假 → 整个条件为假, 退出循环
3 (3)  if y > 1 goto (5)      ; B2 为真 → 条件为真, 进入循环体
4 (4)  goto (14)             ; B2 为假 → 退出循环
5 (5)  t1 := - n              ; 单目负 uminus
6 (6)  t2 := t1 + 2
7 (7)  t3 := x * y
8 (8)  if t2 < t3 goto (10)   ; 内层 if 的条件
9 (9)  goto (12)              ; 条件为假 → else 分支
10 (10) a := 100              ; then 分支
11 (11) goto (1)              ; 循环体结束, 回到 while 头
12 (12) b := a                ; else 分支
13 (13) goto (1)              ; 回到 while 头
```

1.2 带注释语法树



2 习题 10.2(2)

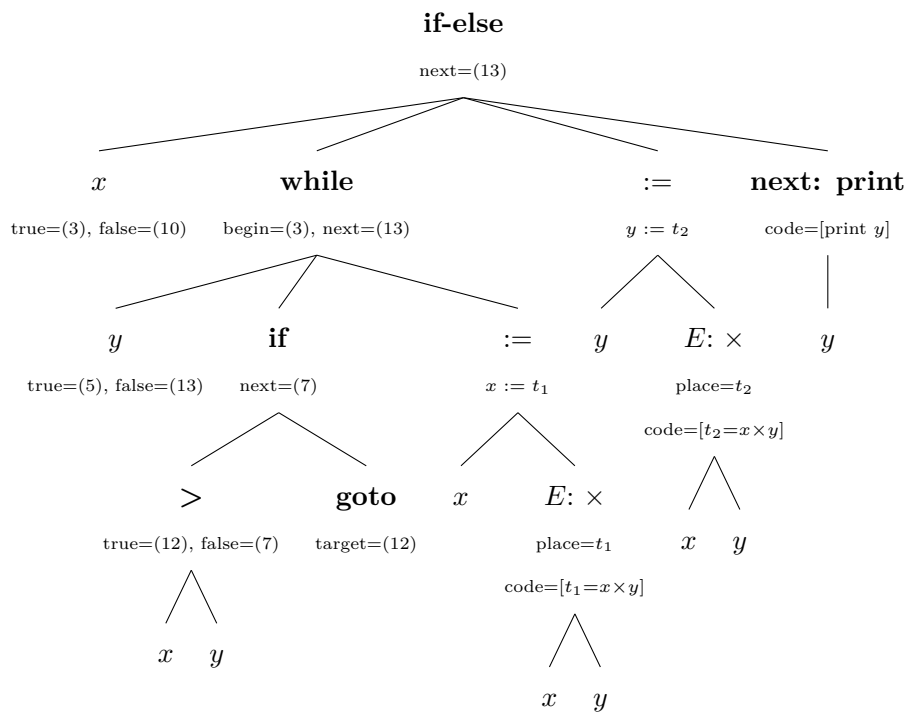
2.1 三地址代码

```

1 (1) if x goto (3)           ; B 为真, 进入 while
2 (2) goto (10)              ; B 为假, 进入 else
3 (3) if y goto (5)          ; while 条件
4 (4) goto (13)              ; 退出整个 if-else
5 (5) if x > y goto (12)      ; 跳到 next
6 (6) goto (7)               ; 否则执行赋值
7 (7) t1 := x * y
8 (8) x := t1
9 (9) goto (3)               ; 回到 while 头
10 (10) t2 := x * y
11 (11) y := t2
12 (12) print y              ; next 标号处

```

2.2 带注释语法树



3 习题 10.3

设属性含义如下：

属性	含义
$S.code$	S
$S.begin$	
$S.next$	
$B.true$	
$B.false$	

对产生式

$$S \rightarrow \text{for}(S_1; B; S_2)S_3$$

设计语义规则如下：

$$\begin{aligned} S.code &= S_1.code \\ &\quad B.code \\ &\quad S_3.code \\ &\quad S_2.code \\ &\quad \text{goto } B.begin \end{aligned}$$

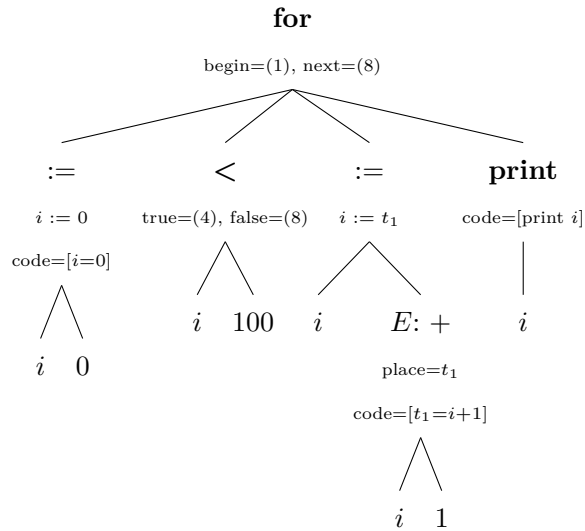
其中：

$$\begin{aligned} B.begin &= \text{条件判断的入口地址} \\ B.true &= S_3.begin \\ B.false &= S.next \\ S_2.next &= B.begin \end{aligned}$$

三地址代码为：

```
1 (1) i := 0 ; S1.code, 初始化
2 (2) if i < 100 goto (4) ; B 为真, 进入循环体
3 (3) goto (8) ; B 为假, 退出 for
4 (4) print i ; S3.code, 循环体
5 (5) t1 := i + 1 ; S2.code, 计算 i+1
6 (6) i := t1 ; 更新 i
7 (7) goto (2) ; 回到条件判断
```

带注释语法树：



4 习题 10.5

程序中的声明如下：

```

1  int x; float z;
2
3  int a[10,20], b[6];
4
5  float bar(int brr[]) {
6      float x;
7      x = brr[0] + brr[5];
8      return x;
9  }
10
11 float foo(int x; float boo(nil); int arr[]) {
12     if (x == 0) z = sqrt(boo(arr[0],));
13     else return boo(arr[x],);
14 }
  
```

设类型宽度如下：

类型	宽度
int	4
float	8
数组名或函数形参地址	4

采用全局名表示法，名字的形式为：

作用域名@局部名

全局作用域用 @ 表示。

@table

名字	类型	相对地址
@x	int	0
@z	float	4
@a	array(10, array(20, int))	12
@b	array(6, int)	812
@bar	function(int[]) → float	—
@foo	function(int, float(nil), int[]) → float	—

@code

```
1 @code:
2 // 全局变量声明只填写符号表，不产生可执行三地址代码
```

bar@table

名字	类型	相对地址
bar@brr	int []	0
bar@x	float	4

bar@code

```
1 bar@code:
2 (1) t1 := bar@brr[0] ; 取 brr[0]
3 (2) t2 := bar@brr[20] ; 取 brr[5], 偏移量为 5*4=20
4 (3) t3 := t1 + t2 ; 计算 brr[0]+brr[5]
5 (4) bar@x := inttofloat(t3) ; 赋给局部 float 变量 x
6 (5) return bar@x ; 返回 x
```

foo@table

名字	类型	相对地址
foo@x	int	0
foo@boo	function(nil) → float	4
foo@arr	int []	8

foo@code

```

1  foo@code:
2  (1)  if foo@x == 0 goto (3)      ; 判断 x==0
3  (2)  goto (10)                  ; 否则进入 else 分支
4
5  (3)  t1 := foo@arr[0]            ; 取 arr[0]
6  (4)  param t1                   ; 传给 boo
7  (5)  t2 := call foo@boo, 1       ; 调用 boo(arr[0])
8  (6)  param t2                   ; 传给 sqrt
9  (7)  t3 := call sqrt, 1          ; 调用 sqrt
10 (8)  @z := t3                   ; 赋给全局变量 z
11 (9)  goto (15)                  ; 退出 if-else
12
13 (10) t4 := foo@x * 4             ; 计算 arr[x] 的偏移量
14 (11) t5 := foo@arr[t4]           ; 取 arr[x]
15 (12) param t5                   ; 传给 boo
16 (13) t6 := call foo@boo, 1       ; 调用 boo(arr[x])
17 (14) return t6                  ; 返回结果
18
19 (15) return

```

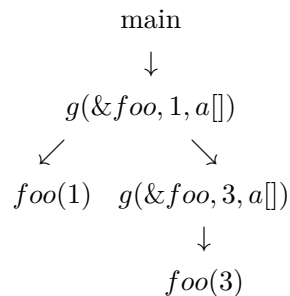
5 习题 11.2

已知数组初始化为 $a[i, j] = j - i$ ，所以有 $a[0, 2] = 2$, $a[1, 0] = -1$ 。按行优先存放时, $b[3] = a[1, 0] = -1$ ，因此 $b[3]$ 在条件判断中为真。

主程序中先有 $x = 1$ ，然后调用 $g(\&foo, x, a[])$ ，所以第一次调用为 $g(\&foo, 1, a[])$ 。此时 $r = \&foo$, $y = 1$, $b = a$ 。因为 $1 \leq x$ 且 $b[3]$ 为真，所以执行 $y = g(\&r, r(x), b[])$ 。其中 $r(x) = foo(1) = 1 + a[0, 2] = 3$ ，因此又调用 $g(\&foo, 3, a[])$ 。

第二次调用 $g(\&foo, 3, a[])$ 时, $y = 3$ ，而全局 $x = 1$ ，所以 $3 \leq 1$ 为假，执行 else 分支: $y = r(y) = foo(3)$ 。又因为 $foo(3) = 3 + a[0, 2] = 5$ ，所以 $foo(3)$ 的返回值为 5。

活动树



各活动记录

活动	活动记录主要内容
$g(\&foo, 1, a[])$	$r = \&foo, y = 1, b = \&a[0]$
$foo(1)$	$x = 1 \rightarrow 3$, 返回值 = 3
$g(\&foo, 3, a[])$	$r = \&foo, y = 3, b = \&a[0]$
$foo(3)$	$x = 3 \rightarrow 5$, 返回值 = 5

foo(3) 结束但尚未释放时的栈快照

设栈向低地址方向增长，栈底单元地址为 500。此时 $foo(1)$ 已经返回并释放，栈中还保留 $main$ 、第一次 g 、第二次 g 和 $foo(3)$ 。

地址	内容	说明
500	main 活动记录	栈底
499	$r = \&foo$	第一次 g
498	$y = 1$	第一次 g
497	$b = \&a[0]$	第一次 g
496	返回地址	返回 main
495	动态链 = 500	指向 main
494	$r = \&foo$	第二次 g
493	$y = 3$	第二次 g
492	$b = \&a[0]$	第二次 g
491	返回地址	返回第一次 g
490	动态链 = 499	指向第一次 g
489	$x = 5$	$foo(3)$
488	返回值 = 5	$foo(3)$
487	返回地址	返回第二次 g
486	动态链 = 494	指向第二次 g

所以在 $foo(3)$ 活动结束后还未释放时，栈顶为地址 486，返回值为 5。

6 习题 11.4

$\varepsilon@table$

名字	类型	说明
$\varepsilon@g$	$\text{int}(\text{int}, \text{int}(\text{nil}), \text{int}[])$	全局函数
$\varepsilon@f$	$\text{void}()$	全局过程

$\varepsilon@code$

```
1 epsilon@code:
2 (1) call epsilon@f, 0      ; 执行 f()
```

$\varepsilon@label$

标号	含义
$\varepsilon@g$	g 的入口地址
$\varepsilon@f$	f 的入口地址

$g@table$

名字	类型	说明
$g@x$	int	形参
$g@h$	int(nil)	过程形参
$g@b$	int[]	数组形参
$g@y$	int	局部变量

$g@code$

```
1 g@code:
2 (1) g@y := g@x + 1      ; y = x + 1
3 (2) t1 := g@x * 4       ; 计算 b[x] 的偏移量
4 (3) g@y := g@b[t1]      ; y = b[x]
5 (4) param g@y           ; 调用 h(y)
6 (5) t2 := call g@h, 1
7 (6) g@y := t2
8 (7) return g@y          ; return y
```

$g@label$

标号	含义
$g@entry$	g 的入口地址
$g@return$	g 的返回地址

f@table

名字	类型	说明
<i>f@r</i>	<code>int(int)</code>	<i>f</i> 中定义的局部过程
<i>f@c</i>	<code>array(2, int)</code>	局部数组
<i>f@x</i>	<code>int</code>	局部变量
<i>f@v</i>	<code>int</code>	局部变量

f@code

```

1 f@code:
2 (1)  f@c[0] := 11          ; c[0] = 11
3 (2)  f@c[4] := 22          ; c[1] = 22
4 (3)  f@x := 0              ; x = 0
5 (4)  param f@x             ; 实参 x
6 (5)  param &f@r            ; 过程 r 的入口地址
7 (6)  param f@c             ; r 的环境信息
8 (7)  param f@c             ; 数组实参 c[]
9 (8)  t1 := call epsilon@g, 4
10 (9)  f@v := t1             ; v = g(x,&r,c[],c[])
11 (10) t2 := f@v + 5
12 (11) return t2            ; return v + 5

```

f@label

标号	含义
<i>f@entry</i>	<i>f</i> 的入口地址
<i>f@return</i>	<i>f</i> 的返回地址
<i>f@r</i>	<i>r</i> 的入口地址

r@table

名字	类型	说明
<i>f@r@y</i>	<code>int</code>	<i>r</i> 的形参

r@code

```

1 r@code:
2 (1)  t1 := f@r@y + 2      ; y + 2
3 (2)  return t1           ; return y + 2

```

$r@label$

标号	含义
$r@entry$	r 的入口地址
$r@return$	r 的返回地址